1. The problem: one mistake eats 100 correct answers

Contracts, forms and scans are typed into systems by hand. Can a model book the fields itself?

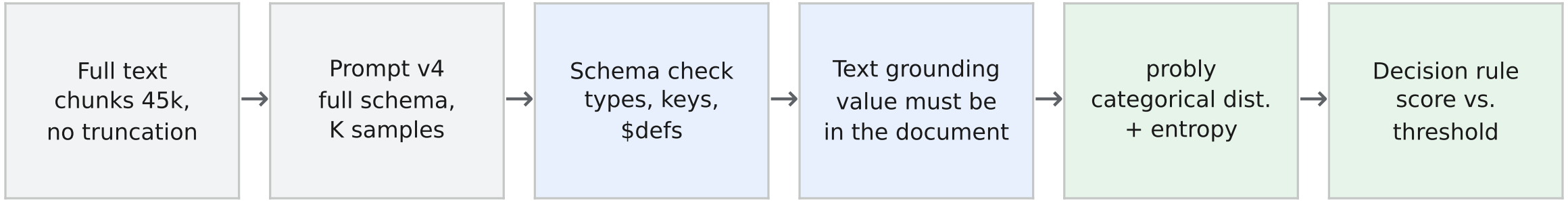
Human review cost 1 per field	**Wrong booked value cost 100**	**Break-even precision > 99.0 %**

- Net saving = (accepted − 100 × (wrong + hallucinated)) / all fields. At error cost 20 the bar is 95.0%, at 500 it is 99.8%.
- The bar to beat is the human baseline: `never` answers nothing and scores exactly 0.0%.
- A careless system loses money: the most-common-value baseline is right in 60% of its answers and scores -177.8%.
- 57.6% of all fields are absent (null). 70.6% are contract clauses that must match the gold text. About 40% of the form PDFs are scans without a text layer.
- So the real task is not extraction. It is knowing when not to answer.

numbers from results/*.json, official scorer 5_score.py

2. Architecture: extract, verify, quantify, decide

Every stage can only remove a value. Nothing is accepted without evidence.



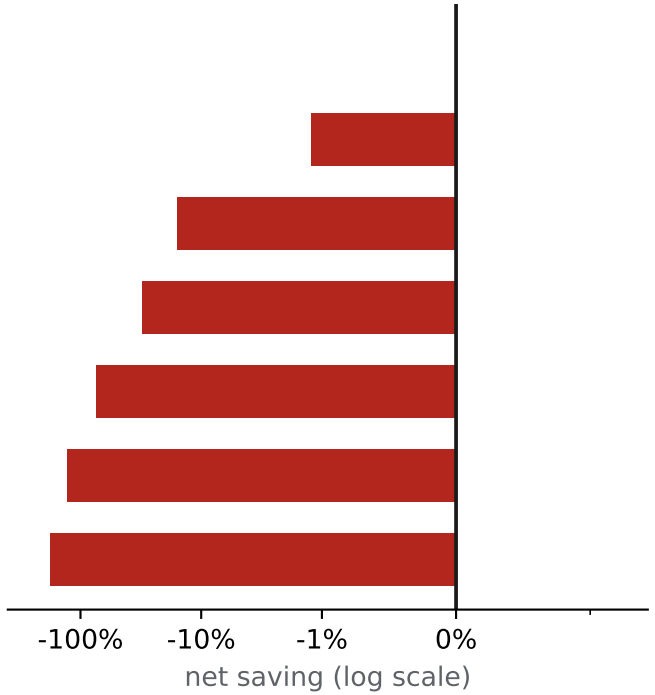
- Scan or broken PDF: the whole document is null, no request is sent.
- probly: the K answers of a field are clustered by exact value. The cluster frequencies form a categorical distribution. Its entropy (nats, normalised by log K) measures how much the samples disagree.
- Four named decision rules: k1_grounded (model confidence), k_agreement_only, k_agreement_conf (min of agreement and confidence), k_entropy (1 − H / log K).
- Freeze and audit: method and threshold are chosen on dev_select, frozen, and applied once to dev_audit. Only a positive audit may switch the system on.
- Submitted decision: all-abstain (abstain_all = true, threshold 1.0). 0 requests, net 0.0%.

numbers from results/*.json, official scorer 5_score.py

3. Result: low coverage, still below the human baseline

Net saving at error cost 100, threshold 0.99. `never` (human reviews everything) = 0.0%.

Variant	Net saving	Precision	Coverage	Accepted / errors
never / all-abstain (submitted) all	**0.0%**	n/a	0.0%	0 / 0
probly pipeline, dev 78 of 540 docs	**-1.2%**	97.0%	1.4%	132 / 4
probly pipeline, test_public 40 of 176 docs	**-15.7%**	93.7%	7.2%	205 / 13
LLM confidence only, dev 307 of 540 docs	**-30.9%**	92.0%	10.4%	976 / 78
probly K=3 frozen, dev_audit 7 docs	**-73.8%**	75.0%	9.1%	4 / 1
LLM confidence only, test_public 75 of 176 docs	**-129.4%**	88.9%	31.0%	892 / 99
most common value, test_public (0.95) all	**-177.8%**	60.2%	11.1%	319 / 127



- probly cuts the loss from -30.9% to -1.2% on dev, but coverage falls to 1.4% and precision is 97.0%, not 99%. Every active variant is below 0.0%.
- Why coverage is low: strict text grounding, threshold 0.99, scans abstained, and 0 accepted CUAD clause fields. Partial runs are scored against the full split, so their percentages are diluted.
- Error cost 20: probly on dev is +0.2%. Error cost 500: -8.4%. All-abstain is never negative. That is why we submit it.

numbers from results/*.json, official scorer 5_score.py

4. One real error, and the five last fixes

dev_audit, field labor_categories (a table): three samples agreed, every cell was in the document.

Schema wants
labor_category_title, year_1_rate: number

Model returned (3 of 3 samples)
labor_category, year_1: "$241.81"

accepted at confidence 0.99, wrong: cost 100

- This one field turned the frozen choice from +8.5% (dev_select) into -73.8% (dev_audit, 4 accepted, 1 wrong).
- Agreement is not correctness: k_entropy and k_agreement_only score -1435% on the audit at 0.95. The model repeats the same mistake three times.
- With schema validation the field is invalid (post-hoc on the same samples: 3 accepted, 0 wrong). Too few fields to prove anything.

Five last fixes (32 tests, not yet run on the API)

a transport.py

Line breaks and tabs inside JSON strings are accepted. Long clauses are no longer lost.

b requests.py, prompt v4

Booleans as JSON true/false. Multi-part clauses in one string joined with " | ".

c grounding.py

Supplier’s = Supplier's. Curly quotes, dashes and non-breaking spaces are unified.

d decision.py

A value with confidence 0.0 becomes null. Nothing is accepted without confidence.

e uq_probly.py

status_counts per field in the diagnostics: why a field was rejected.

numbers from results/*.json, official scorer 5_score.py

5. Limits, and the levers to get above the human baseline

The gap is concrete: 132 accepted fields with 4 errors. With at most 1 error the pipeline would be positive.

Limits

- The submitted system saves nothing (0.0%). It only avoids losing money.
- Best active precision 97.0%, needed > 99%. Coverage 1.4%, zero for contracts.
- Prompt v4 and the five fixes are not measured on the API.
- Runs cover a part of the data. The audit set has 7 documents.
- Confidence is not calibrated: at 1.00 the plain LLM is still wrong in 9.5% of its answers.
- No OCR or vision. Tested on Windows only.

Levers for a positive net saving

- Measure prompt v4 and the fixes on unused dev documents. Contract clauses (70.6% of the fields) become acceptable for the first time.
- Vision or OCR for image-only scans (about 40% of the forms), with their own stricter threshold.
- Calibrate the probability on several hundred labelled dev candidates (probly calibrator; features: confidence, agreement, entropy, status_counts). Choose the threshold with a conformal risk bound.
- Field-specific instead of document-wide abstention: accept only field types with proven precision.
- A second yes/no check that an accepted clause really belongs to its field.
- Until an independent audit is positive: suggest mode. Pre-fill with evidence, the human confirms.

numbers from results/*.json, official scorer 5_score.py